

Module 2 - Antimicrobial Resistance from WGS V2

Detecting and interpreting AMR determinants in Salmonella Typhi assemblies

Table of contents

Introduction	4
1 Module setup	5
1.1 Learning objectives for the module	5
1.2 The input dataset	5
1.3 The module at a glance	6
1.4 Software environment	6
1.5 Working directory layout	7
1.6 Verifying the tools	8
1.7 A note on the AMR databases	9
1.8 Check your understanding	9
2 Background	10
2.1 Three main mechanisms of resistance	10
2.2 Acquired, intrinsic, and adaptive resistance	11
2.3 Genotypic vs phenotypic resistance	11
2.4 Common AMR determinants in <i>S. Typhi</i>	12
2.5 Check your understanding	12
3 Resistance interpretation with CARD and RGI	13
3.1 What is CARD?	13
3.2 RGI's detection model types	14
3.3 RGI hit categories: Perfect, Strict, Loose	14
3.4 Loading CARD	15
3.5 Running RGI on one assembly	15
3.6 Reading the RGI output	16
3.7 RGI heatmap across all samples	17
3.8 Exercises	17
3.9 Check your understanding	18
3.10 Further reading	19
4 NCBI AMRFinderPlus	20
4.1 Why a second tool?	20
4.2 AMRFinderPlus design	20
4.3 Running AMRFinderPlus on one assembly	21
4.4 Reading the AMRFinderPlus output	21
4.5 Running on all samples	22
4.6 Updating the AMRFinderPlus database	23
4.7 Check your understanding	24

5	Reconciling outputs with hAMRonization	25
5.1	What is hAMRonization?	25
5.2	Why a PHA4GE specification at all?	26
5.3	Harmonising one sample	26
5.4	Loop over all samples	27
5.5	The summary report	27
5.6	Exercises	28
5.7	Check your understanding	28
5.8	Further reading	29
6	Plasmid identification with PlasmidFinder	30
6.1	Plasmids, replicons, and incompatibility groups	30
6.2	How PlasmidFinder works	31
6.3	Running PlasmidFinder on one assembly	31
6.4	Running on all samples	32
6.5	Cross-referencing replicons against AMR genes	33
6.6	Exercises	33
6.7	Check your understanding	34
6.8	Further reading	34
7	Wrap-up and discussion	35
7.1	What we built today	35
7.2	Some point to consider during future implementation	35
7.3	Short summary quiz	36
7.4	What can you do next	36
7.5	A few last practical points:	36

Introduction

This module is a hands-on introduction to antimicrobial resistance (AMR) detection from whole-genome sequencing data. We continue from where Module 1 ended: the *Salmonella enterica* serovar Typhi assemblies you produced with shovill and characterised with QUAST. In this module we will investigate which AMR determinants they carry, and on what genetic elements do those determinants sit.

AMR is a major public health issue. In the case of typhoid fever, the H58 lineage that is the globally dominant Typhi genotype is characterised by multidrug resistance (resistance to chloramphenicol, co-trimoxazole and ampicillin), as well as extensive resistance to fluoroquinolones, and most recently the third-generation cephalosporins (extensively drug resistant, XDR). Additionally, occasional resistance to azithromycin, one of the last few oral alternatives, has been reported in Asian countries. Each of those phenotypes is visible in the genome, either as a chromosomal point mutation or as an acquired gene carried on a mobile element. Routine sequencing of clinical isolates is therefore an increasingly central piece of typhoid surveillance.

In this module we will run genotypic AMR tools, RGI (against the CARD database), AMRFinderPlus (NCBI's curated catalogue), and PlasmidFinder (Center for Genomic Epidemiology), as well as use hAMRonline to bring their outputs into a single comparable table. By the end of the day you should be able to take a Typhi assembly, produce a defensible per-sample AMR and plasmid-replicon profile, and have a clear sense of what those profiles do and do not predict about the isolate's clinical phenotype.

1 Module setup

By the end of this module you will have run the following tools: **RGI** → **AMRFinderPlus** → **hAMRonization** → **PlasmidFinder** on the *Salmonella* Typhi assemblies produced in Module 1, and you will be able to interpret the outputs of each step and reason about what the genome data does and does not say about resistance phenotype.

1.1 Learning objectives for the module

By the end of this module, you will be able to:

- Describe the three main routes to AMR (modifications of the antimicrobial molecule, changes in target sites, efflux/uptake change) and acquired (horizontal gene transfer) vs chromosomal mutation-driven resistance.
- Identify AMR genes in bacterial assemblies with **RGI** against the **CARD** database, and interpret results.
- Identify AMR determinants with **NCBI AMRFinderPlus**, including organism-specific point mutations relevant to *Salmonella* Typhi.
- Collate outputs across tools using **hAMRonization** and the PHA4GE AMR Detection Specification.
- Identify plasmid replicon types in assemblies with **PlasmidFinder**, and cross-reference replicon hits with AMR-gene hits to flag mobilisable resistance.
- Explain the principal limitations of genotype-based AMR prediction (silent genes, novel mechanisms, expression, regulatory effects).

1.2 The input dataset

We continue with the same *Salmonella* Typhi isolates from Module 1. The primary input for this module is the assembly produced by shovill — one **contigs.fa** per sample in the **results/assemblies** directory.

Make sure you have all assemblies together in one directory (the ones you ran yourself and the provided ones).

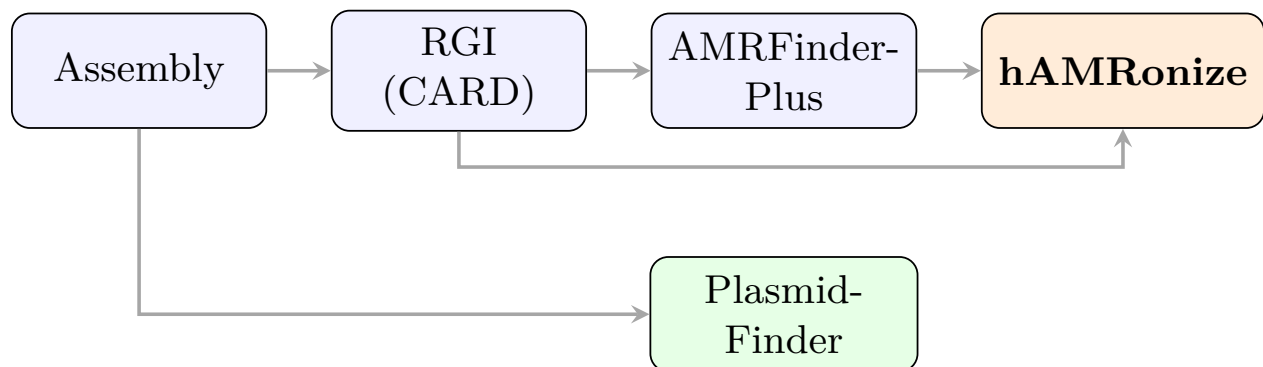
Exercise 1.1 — Check the inputs are in place

From the workshop working directory:

1. List the contents of `results/assemblies/` and confirm that each sample has its own subdirectory.
2. For one sample, confirm that the assembly file is `results/assemblies/<ACC>/contigs.fa` and contains FASTA records (`grep -c '>' results/assemblies/<ACC>/contigs.fa` should return a small positive integer).
3. Note that AMR tools in this module will be run on the assembled contigs, not the raw or trimmed reads. The host-filtered FASTQs are kept as a fallback for tools (such as `ariba` or the read-based mode of `RGI`) that we will mention but not run.

1.3 The module at a glance

We will take Teach assembly through three independent genotypic tools and one harmoniser, plus the plasmid replicon search at the end:



RGI and AMRFinderPlus run independently on the same assembly and produce overlapping but not identical AMR gene calls. hAMRonization parses both outputs and produces one normalised table per sample, plus an across-sample summary. PlasmidFinder runs in parallel and is cross-referenced with the AMR calls in the final exercises.

1.4 Software environment

All tools for this module are in a conda environment called `typhi-amr`.

Activate the workshop environment:

CLI

```
conda activate typhi-amr
```

Your shell prompt should change to indicate the environment is active, typically by adding (typhi-amr) at the front:

```
(typhi-amr) user@host:~/typhi-workshop$
```

If the prompt does not change and you see *command not found* errors, conda may not be initialised for your shell. Speak to an instructor if this is the case and try the `conda activate` command again.

1.5 Working directory layout

Working in the same directory tree from Module 1, create subdirectories to work in:

```
~/typhi-workshop/
Typhi_fastq/          # Module 1 raw reads
results/
  assemblies/         # Module 1 output, input for Module 2
  rgi/                 # to be created
  amrfinder/           # to be created
  hamronization/       # to be created
  plasmidfinder/       # to be created
```

Create the results subdirectories:

CLI

```
mkdir results/rgi results/amrfinder results/hamronization \
results/plasmidfinder
```

1.6 Verifying the tools

Before we start, confirm that every tool we will use today is available in the activated environment. The exact version numbers do not matter for the workshop, but the commands should all run without errors.

💡 CLI

```
rgi main --version
amrfinder --version
plasmidfinder.py -h
hamronize --version
```

Each command should print a single version line (or, for `rgi main --help`, the first few lines of usage). If any command returns *command not found*, the environment is incomplete. Speak to an instructor for installing the missing tools.

As previously mentioned, it is good practice to make a note of the version numbers.

Next, we will download the databases required for RGI and AMRFinderPlus. Make sure you are in the workshop working directory (you can run `pwd` and navigate with `cd` to the right place if needed), and run the following commands:

💡 CLI

```
#download the card database file:
wget https://card.mcmaster.ca/latest/data -O card.tar.bz2
#unpack the file:
tar -xjf card.tar.bz2/card.json
#load the database into a localDB directory:
rgi load -i card.json --local

amrfinder -u #This fetches the AMRFinderPlus DB
amrfinder -l
```

`amrfinder -l` ensures that it is recognised by the tool. Confirm that you can see *Salmonella* among the listed species. The `amrfinderplus` database will have been added to conda env in a path that looks something like this `$CONDA_PREFIX/share/amrfinderplus/data/<dated-folder>/`.

The above commands should have added the `card.json` file and the `localDB` to the directory:

```
~/typhi-workshop/
```

```
Typhi_fastq/          # Module 1 raw reads
card.json             # CARD database (downloaded for RGI)
localDB/              # populated by `rgi load --local`
results/
```

1.7 A note on the AMR databases

You may have noticed that some tools come with a *separate* database that has to be downloaded and kept up to date:

- **CARD** (for RGI): distributed as a single JSON file. After loading, RGI reads from `localDB/`; you can move or delete `card.json` itself if you want.
- **AMRFinderPlus**: distributed as a directory of HMMs and BLAST indices. Downloaded with `amrfinder -u`, which places it inside the active conda environment. AMRFinderPlus finds this automatically.
- **PlasmidFinder**: replicon database is bundled with the conda package and does not need a separate download.
- **hAMRonization**: pure parser, no database.

For your own work after the workshop, the databases must be refreshed regularly: CARD and AMRFinderPlus both release updates every few months. We will look at how to update them later in this module.

1.8 Check your understanding

? Check Your Understanding 1.1

- This module operates on the assembled contigs (`contigs.fa`) rather than the raw or trimmed reads. Name one advantage and one disadvantage of using an assembly as input to AMR detection compared with using reads directly.
- Why do we run two independent AMR tools (RGI and AMRFinderPlus) rather than picking the “best” one? What would using both gain us that using one alone would not?
- Why do CARD and AMRFinderPlus need to be updated regularly, whereas the *Salmonella* Typhi CT18 reference genome we used in Module 1 does not?

2 Background

Genotypic AMR detection from WGS data is in reality a database-search problem: we are asking whether the gene or mutation in the gene under investigation has been previously catalogued as a resistance determinant. To do that, we need to understand what kinds of determinants exist, how they map to clinical resistance, and where the prediction can go wrong.

Antimicrobial resistance is one of the leading global causes of death attributable to infectious disease. WHO's Bacterial Priority Pathogens List (most recently updated in 2024) classifies several Gram-negative enterics, like *Klebsiella pneumoniae*, *Escherichia coli*, *Salmonella* Typhi, as critical or high-priority targets for surveillance and new-drug development.

S. Typhi was susceptible to chloramphenicol when the antibiotic was introduced in the late 1940s; by the 1970s, plasmid-borne chloramphenicol resistance was widespread. The H58 lineage (now known by its GenoTyphi genotype 4.3.1) emerged in South Asia in the late 1990s carrying an IncHI1 multidrug-resistance plasmid encoding resistance to ampicillin, chloramphenicol, and co-trimoxazole simultaneously. From around 2010 onward, the H58 sublineages acquired chromosomal *gyrA* point mutations conferring fluoroquinolone non-susceptibility. In 2016 in Sindh, Pakistan, XDR variant additionally carrying *bla*_{CTX-M-15} (third-generation cephalosporin resistance) and *qnrS* on the same plasmid was isolated.

2.1 Three main mechanisms of resistance

Almost every clinically relevant AMR determinant works through one of three broad strategies.

1. Enzymatic inactivation
degrade or modify drug

2. Target alteration
modify drug's binding/target site

3. Efflux & reduced uptake
pump drug out / keep it out

*e.g. β -lactamases (*bla*-TEM) e.g. *gyrA* SNPs; 23S-rRNA mutations e.g. *acrAB-tolC* pump upregulation*

Enzymatic inactivation. Bacteria produce an enzyme that cleaves, modifies, or sequesters the antibiotic before it reaches its target. The classic example is the β -lactamases (*bla* genes), which hydrolyse the four-membered ring of penicillins, cephalosporins, and carbapenems. These enzymes are encoded by discrete genes. Most are acquired, sitting on plasmids or other mobile elements, and some are intrinsic to specific organisms. Detection on WGS relies on gene presence/absence.

Target alteration. Bacteria change the molecule the drug binds to. An example is fluoroquinolone resistance via point mutations in *gyrA* (the gene encoding the A subunit of DNA gyrase, the drug's target). A single amino-acid change at codon 83 or 87 is enough to substantially raise the MIC of ciprofloxacin. The full *gyrA* gene is present in every *Salmonella* genome; what we are looking for is not gene presence/absence, but a specific SNP within it.

Efflux and reduced uptake. Bacteria pump antibiotics out (efflux pumps such as the AcrAB-TolC system) or close the doors against them (loss of outer-membrane porins such as OmpF). Both mechanisms typically involve regulatory changes rather than gene gain or loss. Resistance here is harder to predict from a static genome alone because expression levels matter, and expression levels are not visible in DNA sequence.

2.2 Acquired, intrinsic, and adaptive resistance

Acquired (genetic) resistance arises when a previously susceptible strain becomes resistant through genetic mutations or horizontal gene transfer (a gene on a plasmid or a transposon), such as β -lactams in Gram-negatives. It is what plasmid-replicon typing is mostly concerned with.

Intrinsic (natural) resistance is the innate ability of a bacterial species to resist and antimicrobial due to its normal structural or functional characteristics. Examples include lacking the target site for the drug, an impermeable outer membrane, or constitutively expressing baseline efflux pumps. As a Gram negative, *Salmonella* is intrinsically resistant to vancomycin, for example. AMR tools generally do not report intrinsic resistance because it is already known from species identity.

Adaptive (environment-triggered) resistance refers to phenotypic resistance that is induced by exposure to the drug but reverts when the drug is removed. Bacteria rapidly alter their gene and protein expression to survive the environmental stress, and revert to their susceptible state once the trigger is removed. Examples include biofilm tolerance, and persister cells. Adaptive resistance is not included in WGS-based prediction.

2.3 Genotypic vs phenotypic resistance

Genotypic resistance is a prediction, not a measurement. Phenotypic resistance, i.e. the actual MIC measured by broth microdilution or disc diffusion, is what determines whether a particular antimicrobial will work against the bacterial strain under study. The genotype is a strong *predictor* of phenotype for many drug-mechanism combinations, but the correlation is not always perfect. The intermediate steps, such as gene expression incl. silent genes, regulatory mutations, post-translational modifications, drug-target interaction, other novel mechanisms or novel/unknown mutations etc., are not captured by WGS, serving as sources of genotype-phenotype mismatch:

- **Silent genes.** The resistance gene is in the genome but not expressed (e.g., promoter mutation).

- **Regulatory mutations** that activate intrinsically present efflux pumps (e.g., *acrR* loss-of-function increasing AcrAB-TolC expression).
- **Novel determinants.** The gene or a SNP is present and conferring resistance, but it is not yet in the database.
- **Background genetic context.** The same *bla_OXA-1* gene can produce a wide range of MICs depending on copy number, neighbouring porin mutations, etc.

Also, another layer is added by the variations in the MICs leading to susceptible, intermediate, or resistant characterisation, defined by EUCAST and CLSI *breakpoints*. Breakpoints may change over time as committees re-evaluate clinical evidence.

2.4 Common AMR determinants in *S. Typhi*

- ***gyrA* S83F or S83Y:** SNP(s) in the chromosomal *gyrA* gene, conferring fluoroquinolone non-susceptibility.
- ***bla_TEM*:** narrow-spectrum β -lactamase, conferring ampicillin resistance. Often on the H58 IncHI1 plasmid.
- ***catA1*:** conferring chloramphenicol acetyltransferase resistance.
- ***sul1*, *sul2*:** dihydropteroate-synthase variants conferring sulphonamide resistance by not being inhibited by the drug.
- ***dhfr*:** dihydrofolate-reductase variants conferring trimethoprim resistance.
- ***bla_CTX-M-15*:** extended-spectrum β -lactamase that is plasmid-borne, conferring resistance to third generation cephalosporins.
- ***qnrS*:** plasmid-mediated gene leading to resistance to fluoroquinolones like ciprofloxacin.

2.5 Check your understanding

? Check Your Understanding 2.1

- Fluoroquinolone resistance in *Salmonella* is mostly caused by point mutations in *gyrA*, whereas third-generation cephalosporin resistance is mostly caused by acquisition of *bla_CTX-M-15*. Why does the difference between these two mechanisms matter for the design of a WGS-based detection tool?
- A clinical *S. Typhi* isolate has the *bla_CTX-M-15* gene present in its assembly, but the laboratory reports the strain as ceftriaxone-susceptible by broth microdilution. Name two distinct biological explanations for this apparent mismatch.
- Why does an AMR detection tool not typically report that *S. Typhi* is resistant to vancomycin, even though every *Salmonella* genome is in fact vancomycin-resistant?

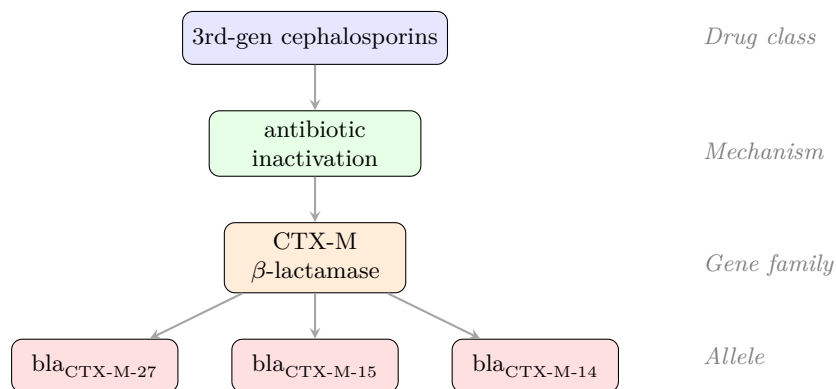
3 Resistance interpretation with CARD and RGI

3.1 What is CARD?

We will work with the **Comprehensive Antibiotic Resistance Database (CARD)**, a curated AMR knowledge base, and the **Resistance Gene Identifier (RGI)**, a tool for searching it. It contains several thousand resistance determinants, each annotated with:

- the **gene sequence(s)** (nucleotide and protein),
- the **resistance mechanism** (one of seven categories: antibiotic inactivation, target alteration, target protection, target replacement, efflux, reduced permeability, antibiotic-target-binding),
- the **drug class(es)** affected,
- the **AMR gene family** the determinant belongs to (e.g., the *CTX-M* family, the *qnr* family),
- and a curated set of **detection models** describing how RGI should decide whether a particular contig matches the entry.

These annotations are nodes in the **Antibiotic Resistance Ontology (ARO)** in which every term (a drug, a mechanism, a gene family, an allele) has a stable identifier and a defined relationship to every other term. This allows for reproducible output, comparable across labs and pipelines.



Every allele is linked upward through gene family → mechanism → drug class. RGI reports all four levels for every hit, making its output very useful for interpretation.

3.2 RGI's detection model types

CARD stores how to search for each entry. There are three model types, and the distinction matters for what RGI can and cannot detect.

Protein homolog models. Used for resistance determinants whose presence alone (regardless of sequence variation) confers resistance; most acquired resistance genes fall here, including most β -lactamases. Detection is by BLAST/DIAMOND against the curated protein with a percent-identity and percent-coverage cutoff per allele.

Protein variant models. Used for chromosomal genes where specific point mutations confer resistance, such as *gyrA* and *parC*. The reference protein is present in every isolate of the species (both susceptible and resistant), but resistance is called when one of the catalogued mutations is present.

rRNA gene variant models. Used for ribosomal-target mutations, e.g., 23S rRNA macrolide resistance. Operates on nucleotide sequence rather than protein.

There is also a small set of **protein overexpression** and **knock-out** models in more recent CARD releases for the efflux/porin-loss class of resistance, but they are difficult to interpret from a static assembly alone.

3.3 RGI hit categories: Perfect, Strict, Loose

For every model, RGI assigns the hit one of three categories:

Category	Threshold	Interpretation
Perfect	100 % identity to a curated CARD reference allele	The exact catalogued allele is present. High confidence call.
Strict	Above CARD's curated per-allele cutoff but not a perfect match	A close variant of a catalogued allele. Likely a real and known resistance determinant.
Loose	Below the cutoff	A possible distant homologue. Likely not a true resistance allele. May be filtered out of downstream reports.

The Loose category is still useful as novel determinants do exist, and a Loose hit can occasionally be the first sign of a previously unreported variant.

3.4 Loading CARD

When running RGI for the first time, we must point to the CARD JSON file. We have provided the file at `ref/card.json`. The `--local` flag stores the loaded database in the current working directory rather than installing it globally.

CLI

```
rgi load -i ref/card.json --local
```

This step takes a few seconds. If you ever want to update to the latest CARD release (no need to do it now), the workflow is:

```
wget https://card.mcmaster.ca/latest/data -O card_latest.tar.bz2
tar -xvf card_latest.tar.bz2
rgi load -i card.json --local
```

3.5 Running RGI on one assembly

The basic invocation for an assembly is `rgi main`. We provide the assembled contigs and ask RGI to predict ORFs and search them against the loaded CARD.

CLI

```
rgi main --input_sequence results/assemblies/ERR4025225/contigs.fa \
  --output_file results/rgi/ERR4025225 --input_type contig \
  --local --clean --n 4
```

- `--input_sequence` is the assembly FASTA.
- `--output_file` is the prefix for output files. RGI will write `<prefix>.txt` and `<prefix>.json`.
- `--input_type contig` tells RGI to first call ORFs with Prodigal, then search them as proteins. Use `protein` if you already have a translated proteome.
- `--local` reads CARD from the working-directory load you did above.
- `--clean` deletes intermediate files at the end (Prodigal calls, BLAST databases).
- `--num_threads 4` parallelises the homology search.

The run will take a couple of minutes per assembly. When it finishes, list the outputs:

```
ls results/rgi/
```

```
ERR4025225.json
ERR4025225.txt
```

The `.txt` file is the human-readable per-ORF summary; the JSON is the structured version that downstream tools (including hAMRonization) read.

3.6 Reading the RGI output

Import the `.txt` into a spreadsheet. Have a look at these columns:

Column	What it means
ORF_ID	Identifier of the ORF on the assembly contig
Cut_Off	Perfect / Strict / Loose
Pass_Bitscore	The curated cutoff this hit was compared against
Best_Hit_ARO	The CARD allele the ORF matches (the gene name you usually care about)
ARO	The numeric ARO identifier (stable across CARD releases)
Model_type	protein homolog / protein variant / rRNA gene variant
SNPs_in_Best_Hit_ARO	For variant models, which resistance-conferring SNPs were observed
Drug Class	One or more drug classes the gene confers resistance to
Resistance Mechanism	One of the seven CARD mechanism categories
AMR Gene Family	The family the gene belongs to
Predicted_DNA / Predicted_Protein	The actual sequence detected

An example you may see (this is not necessarily what you will see in the sample you just ran):

```
contig00012_42  Strict  500  CTX-M-15  3001917  protein homolog model
cephalosporin  antibiotic inactivation  CTX-M beta-lactamase
```

The interpretation: contig 12, the 42nd ORF on it, matches CARD's CTX-M-15 model with a strict-tier hit, encoding a β -lactamase of the CTX-M family that inactivates cephalosporins.

3.7 RGI heatmap across all samples

You can run RGI with the `heatmap` command that takes a directory of JSON outputs and renders an HTML/PNG heatmap with samples on one axis and detected resistance determinants on the other. This serves as a useful summary view for a multi-sample surveillance run.

First run RGI on all samples using a `for` loop:

CLI

```
for assembly in results/assemblies/*/contigs.fa; do
    acc=$(basename "$(dirname "$assembly")"); rgi main \
        --input_sequence "$assembly" --output_file results/rgi/${acc} \
        --input_type contig \
        --local --clean -n 4
done
```

Then build the heatmap:

CLI

```
rgi heatmap --input results/rgi --output results/rgi/heatmap
```

Open `results/rgi/heatmap.png` and explore the output. Yellow represents a perfect hit, teal represents a strict hit, purple represents no hit.

3.8 Exercises

Exercise 2.1 — RGI outputs on individual samples

Open the RGI output of two samples (`results/rgi/<accession>.txt`). Then:

- Count the number of **Perfect** hits, **Strict** hits, and **Loose** hits. (Tip: you can use `awk` for this or manually do it in Excel).
- List the unique **Drug Class** values across all Perfect + Strict hits. How many drug classes are these isolates predicted to be resistant to?
- For one Perfect hit, copy the **Best_Hit_ARO** value and look it up on the CARD website (<https://card.mcmaster.ca>). What is the resistance mechanism, and what drug class is affected?

Exercise 2.2 — Heatmap across all samples

Explore the produce heatmap.

- Looking down each row of the heatmap, are there genes that appear in most or all samples?
- Identify samples with the highest number of distinct drug-class hits and samples with the fewest hits. Do you think these are distinct drug classes, or does one sample carry many hits but all in the same class?
- Are there any genes that appears in only one or two samples? Look up its name in the corresponding TSV row and note its **Drug Class** and **AMR Gene Family**. Speculate on what this might mean in terms of AMR evolution (e.g., acquisition, sequencing artefact, lineage-specific marker)?
- Do you see any group(s) of samples sharing the same multi-gene pattern? This may serve as an indication of the presence of a plasmid that we will explore later.

Exercise 2.3 — Interpret a protein-variant hit

RGI's protein-variant model for **gyrA** is one of the most clinically important for Typhi. In your RGI output for any sample:

- Find the row(s) corresponding to **gyrA** (search the TSV for **gyrA** in **Best_Hit_ARO**).
- Look at the **SNPs_in_Best_Hit_ARO** column — what amino-acid substitutions were observed?
- Cross-reference one of those substitutions against the CARD entry for **gyrA**. What drug class does it confer resistance to, and what is the underlying mechanism?

3.9 Check your understanding

? Check Your Understanding 2.1

- What is the difference between RGI's Perfect and Strict tiers? Is the difference a question of confidence, novelty, or both?
- Most published surveillance reports filter out RGI's Loose hits before reporting. Name one scenario where Loose hits *should* be looked at carefully rather than discarded.
- A colleague suggests “we can skip a database step and just BLAST our contigs against a FASTA of all CARD genes and call anything above 80% identity a hit”. Name two things this would get wrong compared to running RGI properly.

3.10 Further reading

Other tools that search CARD or CARD-like databases that are worth knowing about:

- **abricate**: a lightweight multi-database AMR-gene finder. Can use `--db card`, `--db ncbi`, `--db resfinder`, `--db plasmidfinder` and a few others. Simpler interface than RGI making it very good for quick screening but lacks the protein-variant model support for SNP-based resistance.
- **ariba + CARD**: a very common combination, and it's a useful complement to RGI because it works directly on reads rather than on an assembly. Ariba maps reads to a reference panel using cd-hit clustering + minimap2/bowtie2, then locally assembles each cluster to call variants. Handles low-frequency variants and mixed-allele samples better than assembly-based approaches.
- **SRST2**: reads-based screening using a Bowtie2 + per-allele BLAST hybrid. Includes the bundled ARGannot reference and supports custom DBs. Older but still commonly used.
- **ResFinder**: Bundles PointFinder for chromosomal SNP detection. Web interface at <https://cge.food.dtu.dk/services/ResFinder/>.

4 NCBI AMRFinderPlus

We will now run a second, independent tool against a different curated database, NCBI’s **AMRFinderPlus**, and see if there are any disagreements between this and RGI+CARD. Running two tools is standard practice because the two databases may have different curation philosophies and different blind spots.

4.1 Why a second tool?

CARD and NCBI’s **Reference Gene Catalog** (the database AMRFinderPlus searches) are both curated, but there are some differences. It is worth looking at associated publications like doi: 10.1093/nar/gkac920 and <https://doi.org/10.1038/s41598-021-91456-0>.

- **CARD** tends to be broader and updates regularly every few months. It uses BLAST plus KMC to identify genes in a query sequence. ARGs included in the database must be available in the GenBank repository and increase the MIC in an experimental validation setting (with a few exceptions). As a note, CARD is freely accessible for academic researchers, while commercial users need a license.
- **AMRFinderPlus** and the Reference Gene Catalog are conservative about what counts as a confirmed resistance determinant, and applies strict per-gene cutoffs determined from population data. It uses Hidden Markov Model (HMM) to identify genes in a query sequence. It can now use nucleotide or protein sequence and the “plus” subset includes genes related to biocide and stress resistance, general efflux, virulence, or antigenicity.

A practical consequence is that the two tools agree on most “core” determinants, but may disagree on some determinants. An RGI-only call may be a determinant CARD considers established but NCBI is still evaluating. An AMRFinderPlus-only call is usually a stress-response or virulence gene CARD does not include.

4.2 AMRFinderPlus design

AMRFinderPlus differs from RGI in three important ways.

Two search strategies in parallel. AMRFinderPlus uses HMMs for gene families with high sequence diversity, and BLAST for genes with well-defined per-allele cutoffs. This makes AMRFinderPlus more sensitive for diverse gene families (e.g., the highly variable carbapenemases) at the cost of some computational complexity.

Organism-aware analysis. For a number of pathogens of public-health interest, NCBI has curated organism-specific point-mutation panels. For *Salmonella* this includes the *gyrA*, *parC*, *parE* quinolone-resistance-determining region, *acrR* regulatory mutations, the *ramR* regulatory locus, and a number of others. You activate this with `--organism Salmonella`.

“Plus” — beyond AMR. AMRFinderPlus also reports virulence genes (e.g., Typhi-specific toxin genes like *cdtB*), biocide resistance markers (e.g., *qacE*), and metal-resistance genes (e.g., mercury). These are flagged separately and are useful for surveillance context, although they are not always directly clinically actionable.

4.3 Running AMRFinderPlus on one assembly

We will run a basic AMRFinderPlus command with nucleotide input, the organism flag for point-mutation analysis, and an output path.

CLI

```
amrfinder \  
-n results/assemblies/ERR4025225/contigs.fa --organism Salmonella \  
--threads 4 --output results/amrfinder/ERR4025225.tsv
```

- `-n` indicates a nucleotide (assembly) input. Use `-p` for a protein FASTA instead.
- `--organism Salmonella` activates the *Salmonella*-specific point-mutation panel and applies any organism-specific gene cutoffs. This is the option most likely to be missed, but most important to set to ensure that all relevant mutations are called.
- `--output` is a single TSV file (unlike RGI, which writes both `.txt` and `.json`).

4.4 Reading the AMRFinderPlus output

Open the TSV file in your terminal or in Excel. The output TSV has one row per detected determinant and some key columns:

Column	What it means
Protein identifier	The ORF AMRFinderPlus matched against
Contig id / Start / Stop	Where on the assembly the hit lies

Column	What it means
Element symbol	Curated gene name (e.g., <i>gyrA_S83F</i>)
Element name	Full descriptive name of the hit
Scope	core (a confirmed resistance determinant) or plus (virulence / stress / biocide)
Type / Subtype	AMR , POINT , STRESS , or VIRULENCE , POINT-X for point mutations, BIOCIDE , METAL , etc.
Class / Subclass	Drug class affected
Method	How the hit was made: EXACTX (perfect protein match), BLASTX (BLAST hit above cutoff), PARTIALX (truncated), HMM (HMM-only hit), POINTX (organism-specific point mutation)
% Coverage of reference sequence	Length of the hit relative to the reference
% Identity to reference sequence	Identity of the hit

The **EXACTX** method is roughly analogous to RGI's Perfect tier; **BLASTX** is roughly analogous to RGI's Strict; **PARTIALX** flags a hit that is a real match but is truncated at the end of a contig (worth manually inspecting); **HMM** indicates the hit was found only by HMM and not by BLAST, which usually means a divergent variant of a known family.

Exercise 3.1 — Investigate the output of one sample

- How many **AMR**-scope rows does AMRFinderPlus call?
- How many of those are **EXACTX** (perfect), how many are **BLASTX**, how many are **PARTIALX**?
- What are the unique **Class** values, i.e. how many drug classes is the isolate predicted to be resistant to?

4.5 Running on all samples

We can use a **for** loop to run on all assemblies:

CLI

```
for asm in results/assemblies/*/contigs.fa; do
    acc=$(basename "$(dirname "$asm")")
    amrfinder \
        -n "$asm" --organism Salmonella --threads 4 \
        --output results/amrfinder/${acc}.tsv
done
```

Then concatenate each TSV into a single combined table for quick inspection:

💡 CLI

```
# First, make a file (combined.tsv) to include the header row of a
↳ sample TSV and append an extra column 'Sample'
head -n 1 results/amrfinder/ERR4025225.tsv | sed 's/^/Sample\t/' \
  > results/amrfinder/combined.tsv

#Then, for each TSV, append its rows (without the header) to the
↳ combined file
for tsv in results/amrfinder/*.tsv; do acc=$(basename "$tsv" .tsv);
  tail -n +2 "$tsv" | sed "s/^/${acc}\t/" \
  >> results/amrfinder/combined.tsv
done
```

This combined table is a good input for downstream analysis in Excel or R. Have a look at the file by sorting by different columns, or by sample, to develop an understanding of the data.

Exercise 3.2 — Compare AMRFinderPlus and RGI side-by-side

For the *same* sample, open both `results/rgi/<ACC>.txt` and `results/amrfinder/<ACC>.tsv`.

- List the genes called by **both** tools (be aware that the gene-name spelling can differ, e.g., CTX-M-15 vs blaCTX-M-15).
- List the genes called by **only RGI**. Speculate on why.
- List the genes called by **only AMRFinderPlus**. Are any of them STRESS or VIRULENCE scope (which RGI does not cover at all)?
- For the POINT rows in AMRFinderPlus, does RGI agree on the `gyrA` / `parC` mutations? If not, why might that be?

Do this for at least two samples.

4.6 Updating the AMRFinderPlus database

In your own work after the workshop, refresh the AMRFinderPlus database every few months to incorporate any new releases. You can do this by running the following:

```
amrfinder -u
```

4.7 Check your understanding

? Check Your Understanding 3.1

- a) What is the function of the `--organism Salmonella` flag, and what would you miss if you omitted it?
- b) AMRFinderPlus's **EXACTX** method is roughly analogous to RGI's Perfect tier, and **BLASTX** to RGI's Strict tier. So why would RGI and AMRFinderPlus ever disagree on the same gene in the same assembly? Name two reasons (other than version drift between the databases).
- c) A sample produces an AMRFinderPlus **VIRULENCE**-scope hit but no **AMR**-scope hits. Is this an AMR-relevant finding? Why might it still matter for surveillance?

5 Reconciling outputs with hAMRonization

We now have two sets per sample: an RGI and an AMRFinderPlus output. Comparing them by eye, as you just did, works for one or two samples but does not scale easily. The problem is not just volume; it is schema. Every AMR tool emits its own column names, its own gene-name spelling conventions, and its own hit-quality categories. Pivoting across tools requires normalising those schemas first. This is exactly the problem **hAMRonization** is designed to solve.

5.1 What is hAMRonization?

hAMRonization is a parser library and command-line tool maintained by the Public Health Alliance for Genomic Epidemiology ([PHA4GE](#)). It reads the outputs of a long list of AMR-detection tools (RGI, AMRFinderPlus, ResFinder, abricate, ariba, etc.) and produces a single normalised TSV/JSON that conforms to the **PHA4GE AMR Detection Specification**.

The specification is the work of dozens of public-health bioinformaticians from labs across the world. It defines exactly which fields a normalised AMR call should carry: the gene name in a standard ontology, the resistance mechanism, the drug class, the contig and coordinates of the hit, the percent identity and coverage, the tool and database versions, and so on. Every supported tool's output is mapped onto this same set of columns. Each supported tool has a dedicated parser that maps its native output onto the shared PHA4GE specification. The user sees a single normalised table regardless of which tools produced it.

What hAMRonization *does* and *does not* do:

- It **normalises the schema**, so the same column means the same thing across tools.
- It **records provenance**, so each row carries the tool name, tool version, database name, and database version that produced it.
- It does **not** decide which tool is right. If RGI and AMRFinderPlus both call a gene, you get two rows. If they disagree on a gene name (CTX-M-15 vs blaCTX-M-15), hAMRonization will preserve both spellings.
- It does **not** improve sensitivity or specificity over the underlying tools.

5.2 Why a PHA4GE specification at all?

In the global surveillance context, AMR data flows between labs, countries, etc. If lab A reports CTX-M-15 as “*CTX-M-15*” with a CARD ARO of 3001917 and lab B reports it as “b1aCTX-M-15” with an NCBI gene ID, the same epidemiological event ends up looking like two different findings in an aggregated dataset. The PHA4GE spec is the agreement that everyone serialises into the same field names, the same gene-name vocabulary (typically the NCBI Reference Gene Catalog symbol), and the same provenance metadata, while it aids with reporting.

This matters less for an in-lab analysis where one person ran every tool. It matters a lot when the same isolate’s genome is later analysed by a different group, or pooled with hundreds of others for a multi-country surveillance report. Adopting hAMRization in your own workflow makes your AMR calls interoperable with the global surveillance ecosystem at zero additional analytical cost.

5.3 Harmonising one sample

hAMRization will take an input file and the name of the tool that produced it. Required arguments include the tool version and the database version of the call set (the provenance fields that will be seen downstream).

💡 CLI

```
#Let's do for rgi first:
hamronize rgi \
  --analysis_software_version 6.0.5 \
  --reference_database_version 4.0.1 \
  --input_file_name results/rgi/ERR4025225.txt \
  results/rgi/ERR4025225.txt \
  > results/hamronization/ERR4025225_rgi.tsv

#and now for amrfinderplus:
hamronize amrfinderplus \
  --analysis_software_version 4.2.7 \
  --reference_database_version 2026-05-15.1 \
  --input_file_name results/amrfinder/ERR4025225.tsv \
  results/amrfinder/ERR4025225.tsv \
  > results/hamronization/ERR4025225_amrf.tsv
```

Replace the version strings with whatever your rgi main --version, rgi database --version --local, and amrfinder -V actually print, in case you don’t already have this information at hand.

Open the harmonised TSVs for one of your samples in your terminal or in Excel. You will see multiple PHA4GE fields: `input_file_name`, `gene_symbol`, `gene_name`, `reference_database_name`, `reference_database_version`, `reference_accession`, etc. The RGI-derived row and the AMRFinderPlus-derived row for the same gene now share the same column structure, and you can stack them with `cat`, `sort` and `awk`.

5.4 Loop over all samples

Run both harmonisers for all samples:

💡 CLI

```
for tsv in results/rgi/*.txt; do acc=$(basename "$tsv" .txt);  
    hamronize rgi --analysis_software_version 6.0.5 \  
        --reference_database_version 4.0.1 --input_file_name "$acc" "$tsv" \  
    > results/hamronization/${acc}_rgi.tsv  
done  
  
for tsv in results/amrfinder/*.tsv; do acc=$(basename "$tsv" .tsv);  
    hamronize amrfinderplus --analysis_software_version 4.2.7 \  
        --reference_database_version 2026-05-15.1 \  
        --input_file_name "$acc" "$tsv" \  
    > results/hamronization/${acc}_amrf.tsv  
done
```

Again, make sure that the version strings match the ones you actually used. Also, note that we can use this `for` loop since we ran all samples in one batch, using the same tool and database versions.

5.5 The summary report

hAMRonization also has a `summarize` command that takes a directory of harmonised TSVs and produces a single multi-sample HTML report:

💡 CLI

```
hamronize summarize --summary_type interactive \  
    --output results/hamronization/summary.html \  
    results/hamronization/*.tsv
```

Open `results/hamronization/summary.html`. This is an interactive summary of the results, listing every gene detected with each tool, while it also has some search/filter options. Note that we selected this output version (`--summary_type interactive`) but you can also set it to `tsv` or `json`.

5.6 Exercises

Exercise 4.1 — Inspect the outputs

- Produce a tsv file for the summary report.
- Open the TSV report in Excel (you can also explore the file on the terminal using `awk`, `sort`, etc. if you feel comfortable with it). How many *unique* `gene_symbol` values appear across the whole dataset?
- Filter for rows where `gene_symbol` contains `gyrA`. How many samples have a `gyrA` mutation call? Is the call consistent between RGI and AMRFinderPlus where both tools detected it?
- Filter for rows where `drug_class` contains `BETA-LACTAM`. How many samples carry a resistance determinant? Is it always the same gene?

If you have time and energy after the workshop, you can further explore the output by looking at the overlap between the tools. You can do this with `awk` or in R. Have a look at genes called by both or by only one of the two tools/databases. Think about the following:

- Are there gene families that systematically appear in only one tool's output? Speculate on why (database curation differences, scope differences).
- For genes that were only found by one tool, look these up in CARD and/or NCBI's Reference Gene Catalog. Does one of the databases simply not have the gene? Is the cutoff different?

5.7 Check your understanding

? Check Your Understanding 4.1

- What is the PHA4GE AMR Detection Specification, and why is it useful for AMR data being shared across labs and countries?
- hAMRonization is described as harmonisation, not consensus. What is the difference, and why does it matter for analyses you may do after running hAMRonization?
- Two labs sequence the same isolate and report it through different pipelines (lab A uses abricate against CARD; lab B uses RGI against CARD). Both pipelines successfully detect `bla_CTX-M-15`. After both labs run their outputs through hAMRonization, what specifically

becomes possible that was not possible before?

5.8 Further reading

- The hAMRonization paper: Mendes *et al.*, 2024 doi: <https://doi.org/10.1101/2024.03.07.583950>.
- nf-core/funcscan is a Nextflow pipeline that wraps assembly, multiple AMR tools (AMRFind-erPlus, RGI, etc.), and hAMRonization end-to-end, along with other functionalities.
- The PHA4GE website for more information on standards, metadata, and more.

6 Plasmid identification with PlasmidFinder

The AMR profile of a *S. Typhi* isolate is not complete with *which* resistance genes are present only, but also with *where* they sit in the genome (along with other important genomic markers). A chromosomal SNP can be a stable lineage marker, while a plasmid-borne gene can move horizontally across distantly related strains in a single clinical event. The H58 IncHI1 plasmid is a good example: its introduction in a previously susceptible Typhi background turned isolates to MDR in a single conjugation event.

Next, we will identify the plasmid content of our assemblies using **PlasmidFinder** from the Center for Genomic Epidemiology, and cross-reference the replicon hits against the AMR-gene calls from the previous section.

6.1 Plasmids, replicons, and incompatibility groups

A plasmid is an extrachromosomal, usually circular, autonomously replicating piece of DNA. A defining feature of a plasmid is its replication initiation system, the genes and the origin of replication that allow the plasmid to copy itself independently of the host chromosome.

Two plasmids with the same replication system are usually unable to stably coexist in the same cell: they compete for the same replication machinery and one is lost over a few cell generations. This phenomenon is called *incompatibility* and is the basis for classifying plasmids into Inc groups. Every Inc group is named after a representative plasmid (IncF after the F plasmid, IncH after IncHI, etc.) and contains plasmids that share a replication strategy.

Replicon typing is an important molecular surveillance tool that tracks and classifies bacterial plasmids, and determines which family of plasmid are the genes found on. This can predict how a gene may move, what other genes it tends to travel with, and what its host range is likely to be. For example, IncF plasmids can be found in *Escherichia coli* and *Klebsiella pneumoniae* and are often associated with high-risk clinical isolates as they may carry beta-lactamases and/or carbapenemases. IncN plasmids are known for their broad host range spreading carbapenemases across different Enterobacterales species. IncHI plasmids are associated with *Salmonella* and *Enterobacter* and can carry multiple AMR genes.

For Typhi specifically, the global spread and dominance of H58 is driven by the acquisition of IncHI1 plasmid which confer resistance to chloramphenicol, ampicillin, and trimethoprim-sulfamethoxazole.

If your AMR results show the genes conferring resistance to these antimicrobials co-occurring, a positive IncHI1 PlasmidFinder hit is highly likely.

6.2 How PlasmidFinder works

PlasmidFinder maintains a curated database of replicon sequences and performs a BLAST search of your assembly against that database. Hits above identity and coverage thresholds are reported as replicon calls.

PlasmidFinder does not reconstruct the full plasmid sequence, nor does it tell you which AMR genes are on the plasmid. Also, it does not distinguish between an intact, functional plasmid, and a degraded plasmid integrated into the chromosome. A rep gene hit means the sequence is present; it does not guarantee that an autonomously replicating plasmid is present in the cell. This can be further verified for example by looking at the assemblies.

6.3 Running PlasmidFinder on one assembly

PlasmidFinder comes as `plasmidfinder.py` in the conda environment and operates on FASTA input. The database has already been downloaded for you and is in the `plasmidfinder_db` directory. The Enterobacterales database is the right choice for *Salmonella*.

CLI

```
mkdir -p results/plasmidfinder/ERR4025225

plasmidfinder.py -i results/assemblies/ERR4025225/contigs.fa \
  -o results/plasmidfinder/ERR4025225 \
  -p plasmidfinder_db -d enterobacterales -x
```

- `-i` is the input assembly FASTA.
- `-o` is an output directory.
- `-p` points to the database.
- `-d enterobacterales` restricts the search to the Enterobacterales replicon panel. If non is specified all is used.
- `-x` produces extended output (per-hit details).

After it runs, look at the output directory:

```
ls results/plasmidfinder/ERR4025225/
```

```
data.json
Hit_in_genome_seq.fsa
Plasmid_seqs.fsa
results.txt
results_tab.tsv
results.tsv
tmp/
```

What we are most interested in now is the **results_tab.tsv**, which contains one row per replicon hit, with the family, identity, query/template length, contig name, position in contig, and accession number. For each hit, the contig column can be cross-referenced with AMR results as it tells you which assembly contig the plasmid replicon is on.

6.4 Running on all samples

Let's run a for loop for all samples:

💡 CLI

```
for asm in results/assemblies/*/contigs.fa; do
    acc=$(basename "$(dirname "$asm")")
    mkdir -p results/plasmidfinder/${acc}
    plasmidfinder.py -i "$asm" -o results/plasmidfinder/${acc} \
        -p plasmidfinder_db -d enterobacteriales -x
done
```

For a quick view across all samples, concatenate the **results_tab.tsv** files:

💡 CLI

```
#Write a header row into a tsv file:
echo -e "Sample\tDatabase\tPlasmid\tID\tQueryTemplate\tContig\tPosition"
> results/plasmidfinder/combined.tsv

#for each sample append its results_tab.tsv without its header:
for sample in results/plasmidfinder/*; do acc=$(basename "$sample")
    tail -n +2 "$sample/results_tab.tsv" | sed "s/^/${acc}\t/" \
    >> results/plasmidfinder/combined.tsv
done
```

Now you have a single file listing every replicon hit across the dataset.

6.5 Cross-referencing replicons against AMR genes

Let's have a look at the AMR analysis results and the plasmid analysis results side by side; are there any AMR genes on plasmid contigs? We can use the contig name as a shared label between the two outputs. PlasmidFinder tells us which contigs carry a replicon hit; AMRFinderPlus tells us which contigs carry AMR genes. Any AMR gene whose contig matches a contig with a replicon hit is a good indication that is located on a plasmid contig. For now, you can just open the results TSV files from PlasmidFinder (`results/plasmidfinder/[ACCESSION]/results_tab.tsv`) and AMRFinderPlus (`results/amrfinder/[ACCESSION].tsv`) and cross reference any plasmid contigs to any contigs in the AMR tsv file. Do this for 3 to 4 samples. Do you see any patterns?

While this can be a good initial approach, bear in mind that a contig-level match shows that an AMR gene and a plasmid replicon are on the same assembled fragment. It does not prove the AMR gene is on an autonomously replicating plasmid. Short read assemblies often break plasmids into multiple contigs; conversely, a plasmid replicon may sit on a contig that also contains chromosomal sequence if the assembler misjoined them. Also, a plasmid replicon can be present on the chromosome, with no functional plasmid in the cell. To confirm physical plasmid co-location you would normally use other tools, or perform long read sequencing and assembly.

6.6 Exercises

Exercise 5.1 — Sample replicon profiles

- Build a small table with one row per sample and one column per unique replicon family. Annotate which replicons are in which sample (or the percent identity of the best hit).
- How many samples carry the same replicons? Are there any hits of interest in the context of Typhi?
- Do any of the samples not have any PlasmidFinder hits? If so, does this correlate with the AMR results?

Exercise 5.2 — Co-location of AMR genes and replicons

- Are there any samples with an IncHI1 replicon hit?
- List the AMR genes (from RGI or AMRFinderPlus) that lie on those same contigs.
- Does the gene cluster you observe match published H58 IncHI1 cargo? Any genes that are unexpectedly absent or unexpectedly present?

6.7 Check your understanding

? Check Your Understanding 5.1

- a) Why does replicon typing give you a useful surveillance signal?
- b) Two plasmids belong to the same incompatibility group. What does that mean about their ability to stably coexist in the same bacterial cell, and how does that constrain a cell's plasmid content?
- c) You see a positive IncHI1 hit on a contig but no AMR genes on that same contig. Name two plausible explanations.

6.8 Further reading

The tools below all build on or complement what PlasmidFinder does:

- **PlasmidFinder** web interface where you upload a FASTA file.
- **Bandage**: previously mentioned. The shovill `.gfa` graph viewer is the easiest way to visualise circular plasmid loops in an assembly. Combining PlasmidFinder's output with a Bandage view of that contig's neighbourhood in the graph is a quick way to see whether the plasmid is structurally intact.
- **pMLST**: plasmid multi-locus sequence typing, also from the Center for Genomic Epidemiology, distinguishes sub-types within a replicon family. Run downstream of PlasmidFinder when you need higher-resolution plasmid typing.
- **MOB-suite**: a toolkit for the typing and reconstruction of plasmid sequences from WGS assemblies.
- The PlasmidFinder paper: Carattoli *et al.*, 2014, doi: 10.1128/AAC.02412-14.

7 Wrap-up and discussion

We have just run a genotypic AMR analysis on real surveillance Typhi data, using two independent AMR detection tools, a schema harmoniser, and a plasmid replicon typer.

7.1 What we built today

For each of the *Salmonella* Typhi assemblies, you now have:

- An RGI / CARD call set, with Perfect/Strict/Loose tiers and ARO-linked drug-class and mechanism annotation.
- An AMRFinderPlus call set, with the `--organism Salmonella` point-mutation panel activated and AMR/POINT/STRESS/VIRULENCE element types.
- A harmonised PHA4GE-format table merging both call sets per sample, plus an across-sample summary.
- A PlasmidFinder replicon profile per sample, with some cross-referenced against the AMR gene calls to predict plasmid-borne resistance.

7.2 Some point to consider during future implementation

There are no right answers; the goal is to think about the assumptions and judgement calls you will be making.

1. **The same outliers.** In Module 1 you flagged samples for QC reasons. In this module you have flagged samples for AMR reasons. Did the same samples appear as “interesting” in both modules? What does that pattern, or its absence, say?
2. **Tool disagreement.** RGI and AMRFinderPlus may disagree on some hits in some samples. Walk through some of these disagreements and think about which tool would you trust more, and why? Would the answer change for a clinical report vs an epidemiological report?
3. **A known pattern.** Did you see any known patterns (e.g., IncHI1 and MDR) in any samples? What would the next analytical step be to confirm the lineage assignment?
4. **What the genome did not tell you.** We mentioned several limitations about what you could not detect (e.g., silent genes, novel determinants, etc.) . If a clinician asked you “is this isolate going to respond to antibiotic X?”, how confident a prediction would you be willing to make from the WGS data alone, and what additional evidence would you ask for?

7.3 Short summary quiz

?

- Q1.** What is the difference between RGI's protein homolog and protein variant detection models, and why is the distinction important for detecting fluoroquinolone resistance in *Salmonella*?
- Q2.** What does the `--organism Salmonella` flag do in AMRFinderPlus, and what would you miss if you omitted it?
- Q3.** hAMRonization's is about harmonisation, not consensus. In one or two sentences, what does that mean in practice?
- Q4.** Name two distinct biological reasons that a AMR gene called in an assembly might not translate into a phenotypically resistant isolate.

7.4 What can you do next

Genotypic AMR detection from WGS is one piece of a larger surveillance picture. Next you can consider:

- **Lineage assignment.** For Typhi, **GenoTyphi** assigns each isolate to one of the curated clade definitions in the GenoTyphi scheme. The lineage assignment gives you the genomic epidemiological context for the AMR profile.
- **Phylogenetic placement.** Placing each isolate within a global Typhi tree may tell you whether the AMR profile is the expected one for its phylogenetic position, or whether the isolate is an unusual combination of lineage and AMR profile.
- **Phenotypic AMR data.** Linking the genotypic AMR profile to MIC measurements is the standard for validating the surveillance pipeline and for catching genotype-phenotype mismatches.
- **Plasmid reconstruction.** PlasmidFinder identifies replicons but does not reconstruct whole plasmids. You can use other tools including visualising the assemblies if you need to.
- **Outbreak investigation.** Reference-based variant calling on the same reads can produce SNP distances between isolates. Combining this with the AMR profile can be very useful for outbreak investigation.

The next module will cover some of these steps.

7.5 A few last practical points:

- **Save the summary files properly.** Save them with meaningful names so you can be able to go back to them, as well as sharing them with collaborators.

- **The results/ tree from this module is small** so it is easy to keep almost everything long-term.
- **Re-run the analysis when the databases update.** CARD and AMRFinderPlus release new database versions every few months. A re-run with refreshed databases sometimes reveals determinants that were “Loose” or absent in the previous run.

Thank you for working through the module. Feedback — what worked, what was confusing, what should be added or removed — is gratefully received by the workshop organisers.